

No more animations

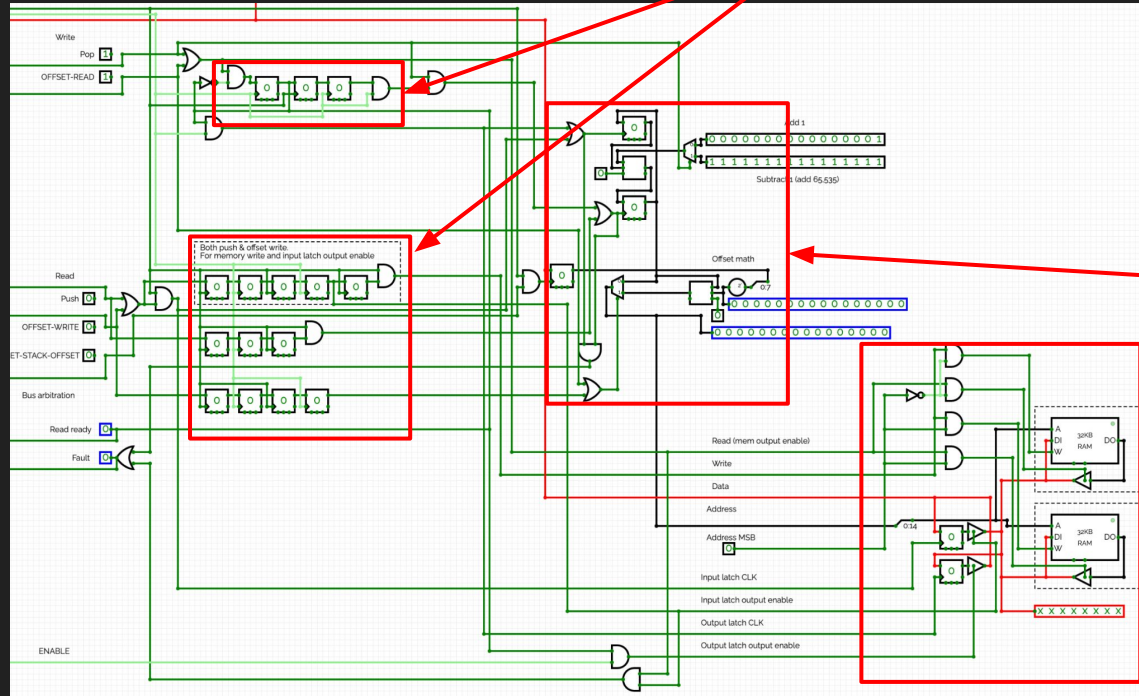
I ran out of time :(

Design process

- Figure out basic parts
- Instruction format
- Most of the logic that is controlled by timing (math, memory)
- Create timing diagrams (Wavedrom)
- Create sequential logic that obeys the timing (Circuitverse)
- Transfer to KiCad
- Design Printed Circuit Boards (PCBs)

Designing it in Circuitverse

Sequencers (timing)



First part I made was the stack (ended up not working)

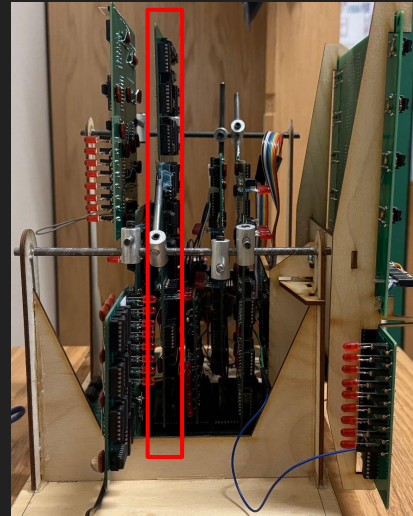
Memory address math

Memory

Arithmetic & Logic unit

0. **ADD** - Add
1. **ADD-C** - Addition carry
2. **NOT** - Bitwise NOT (latch A)
3. **OR** - Bitwise OR
4. **AND** - Bitwise AND
5. **XNOR** - Bitwise XNOR, whether each pair of bits are equal
6. **SHIFT** - Bitshifts A left by the first 3 bits of B
7. **EQ** - Whether bytes are equal
8. **A** - Contents of A latch
9. **B** - Contents of B latch

- Math
- Mostly bitwise operations
- Equality
- Bit Shift



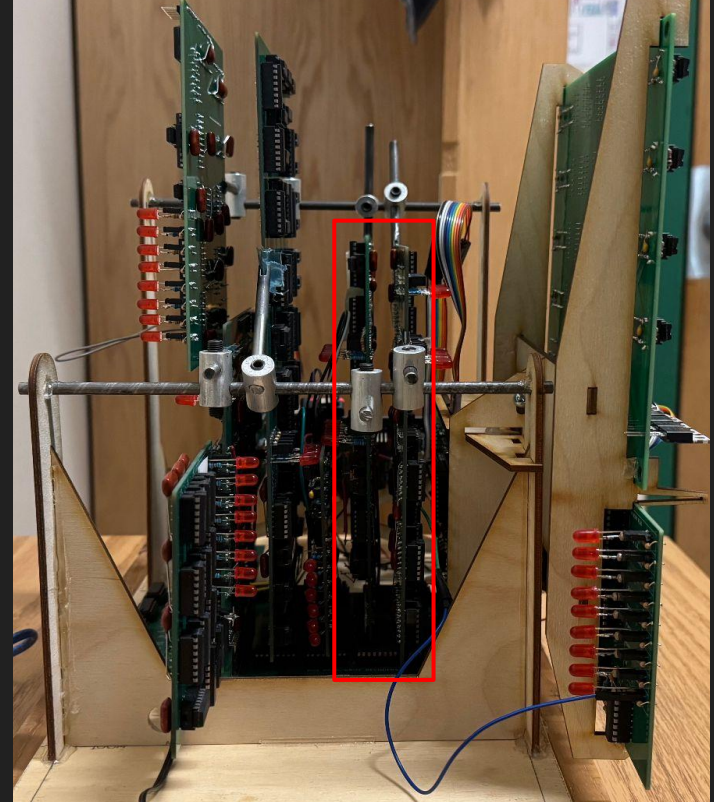
Main Memory: Stack & Heap

Stack: 64 kB

- Push & Pop
- Last In First Out
- Local variables
- Offset read-write

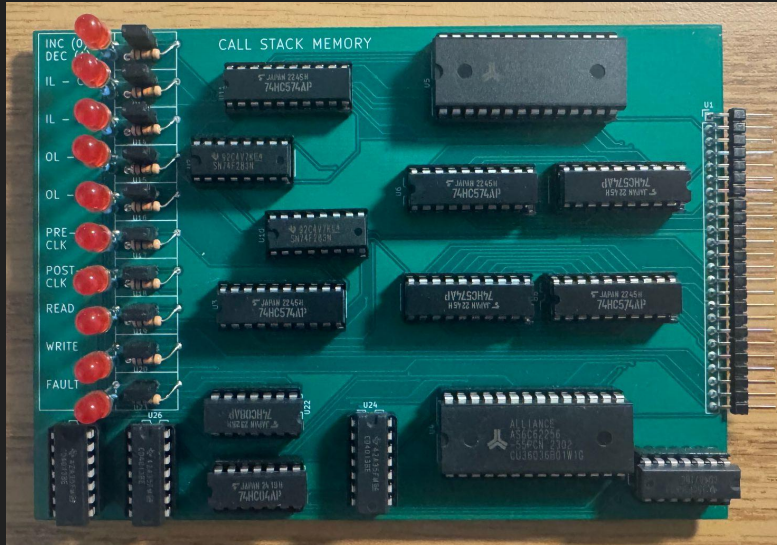
Heap: 64 kB

- Read/Write anywhere
- Global variables



Call Stack

(empty)



PC

Main program

write 0x00 none;# Doing stuff
@call(function_a);# Function call
write 0x01 none;# Doing stuff

@anchor(function_a);
write 0x01 stack-push;
write 0x01 stack-push;
@call(function_add);
write stack-pop gpio-write-a;
return;

@anchor(function_add);
move stack-pop alu-a;
move stack-pop alu-b;
move add alu stack-push;
return;

Call Stack

1. function_a return address



PC

```
# Main program
write 0x00 none;# Doing stuff
@call(function_a);# Function call
write 0x01 none;# Doing stuff

@anchor(function_a);
write 0x01 stack-push;
write 0x01 stack-push;
@call(function_add);
write stack-pop gpio-write-a;
return;

@anchor(function_add);
move stack-pop alu-a;
move stack-pop alu-b;
move add alu stack-push;
return;
```

Call Stack

1. function_a return address



PC

```
# Main program
write 0x00 none;# Doing stuff
@call(function_a);# Function call
write 0x01 none;# Doing stuff
```

```
@anchor(function_a);
write 0x01 stack-push;
write 0x01 stack-push;
@call(function_add);
write stack-pop gpio-write-a;
return;
```

```
@anchor(function_add);
move stack-pop alu-a;
move stack-pop alu-b;
move add alu stack-push;
return;
```


Call Stack

1. function_a return address
2. function_add return address



PC

```
# Main program
write 0x00 none;# Doing stuff
@call(function_a);# Function call
write 0x01 none;# Doing stuff
```

```
@anchor(function_a);
write 0x01 stack-push;
write 0x01 stack-push;
@call(function_add);
write stack-pop gpio-write-a;
return;
```

→ @anchor(function_add);
move stack-pop alu-a;
move stack-pop alu-b;
move add alu stack-push;
return;

Call Stack

1. function_a return address
2. function_add return address



PC

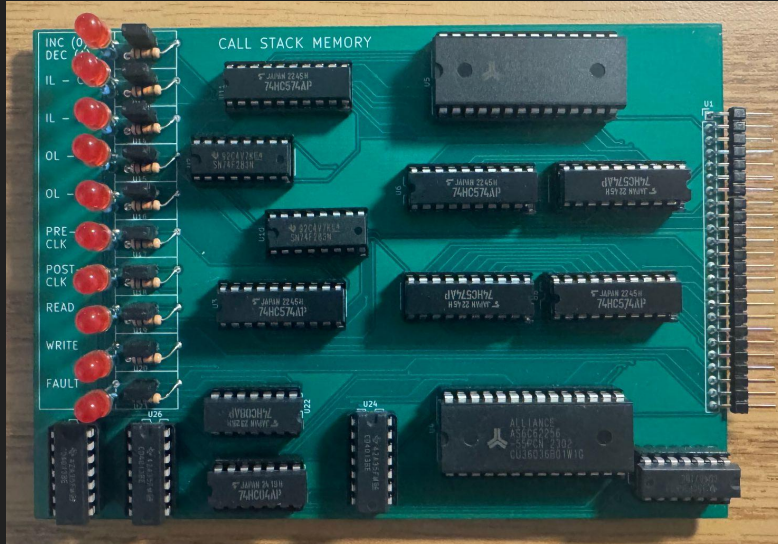
```
# Main program  
write 0x00 none;# Doing stuff  
@call(function_a);# Function call  
write 0x01 none;# Doing stuff
```

```
@anchor(function_a);  
write 0x01 stack-push;  
write 0x01 stack-push;  
@call(function_add);  
write stack-pop gpio-write-a;  
return;
```

```
@anchor(function_add);  
move stack-pop alu-a;  
move stack-pop alu-b;  
move add alu stack-push;  
return;
```

Call Stack

1. function_a return address



PC

```
# Main program
write 0x00 none;# Doing stuff
@call(function_a);# Function call
write 0x01 none;# Doing stuff
```

```
@anchor(function_a);
write 0x01 stack-push;
write 0x01 stack-push;
@call(function_add);
→ write stack-pop gpio-write-a;
return;
```

```
@anchor(function_add);
move stack-pop alu-a;
move stack-pop alu-b;
move add alu stack-push;
return;
```


Call Stack

1. function_a return address



PC

```
# Main program  
write 0x00 none;# Doing stuff  
@call(function_a);# Function call  
write 0x01 none;# Doing stuff
```

```
@anchor(function_a);  
write 0x01 stack-push;  
write 0x01 stack-push;  
@call(function_add);  
write stack-pop gpio-write-a;  
return;
```

```
@anchor(function_add);  
move stack-pop alu-a;  
move stack-pop alu-b;  
move add alu stack-push;  
return;
```

Call Stack

(empty)

Note: Functions can be called from anywhere, even themselves

PC



```
# Main program
write 0x00 none;# Doing stuff
@call(function_a);# Function call
write 0x01 none;# Doing stuff
```

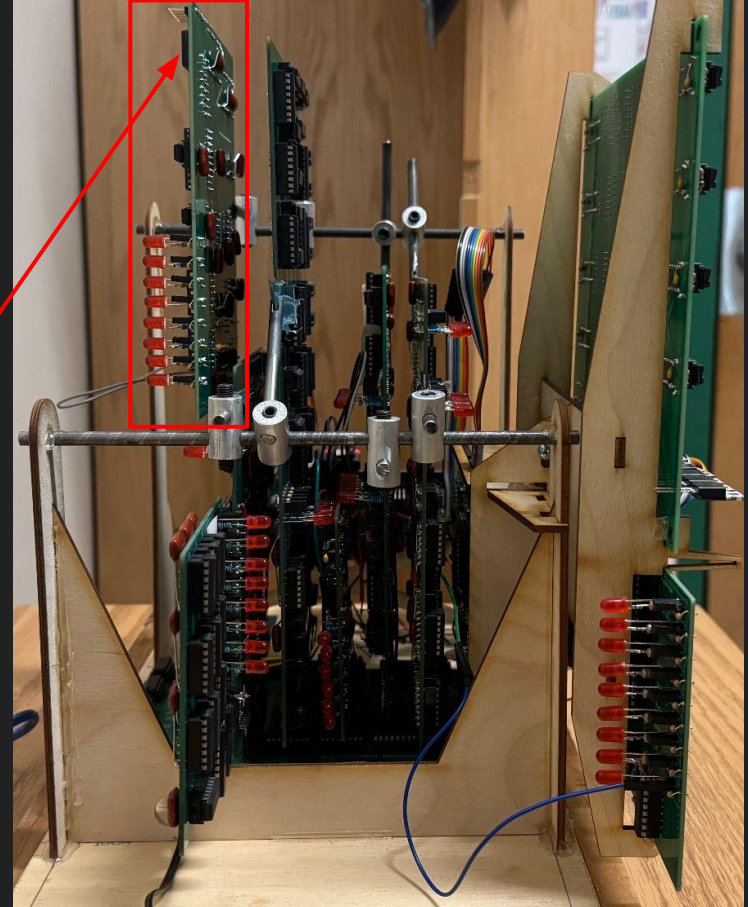
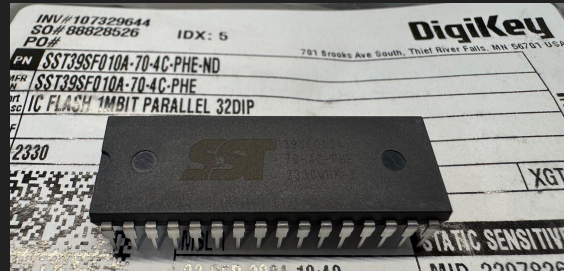
```
@anchor(function_a);
write 0x01 stack-push;
write 0x01 stack-push;
@call(function_add);
write stack-pop gpio-write-a;
return;
```

```
@anchor(function_add);
move stack-pop alu-a;
move stack-pop alu-b;
move add alu stack-push;
return;
```


Program Memory

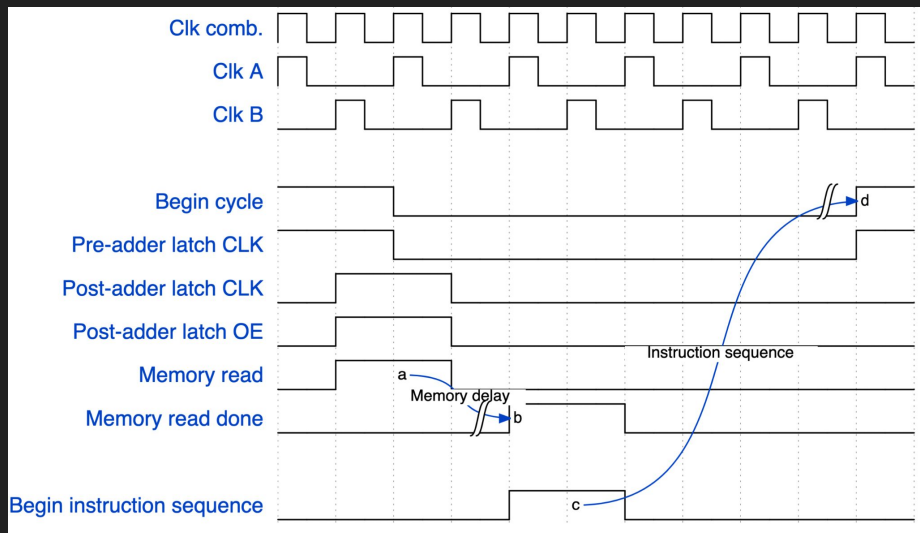
- Non-volatile
- 128 kB
- Program with an Arduino

x2



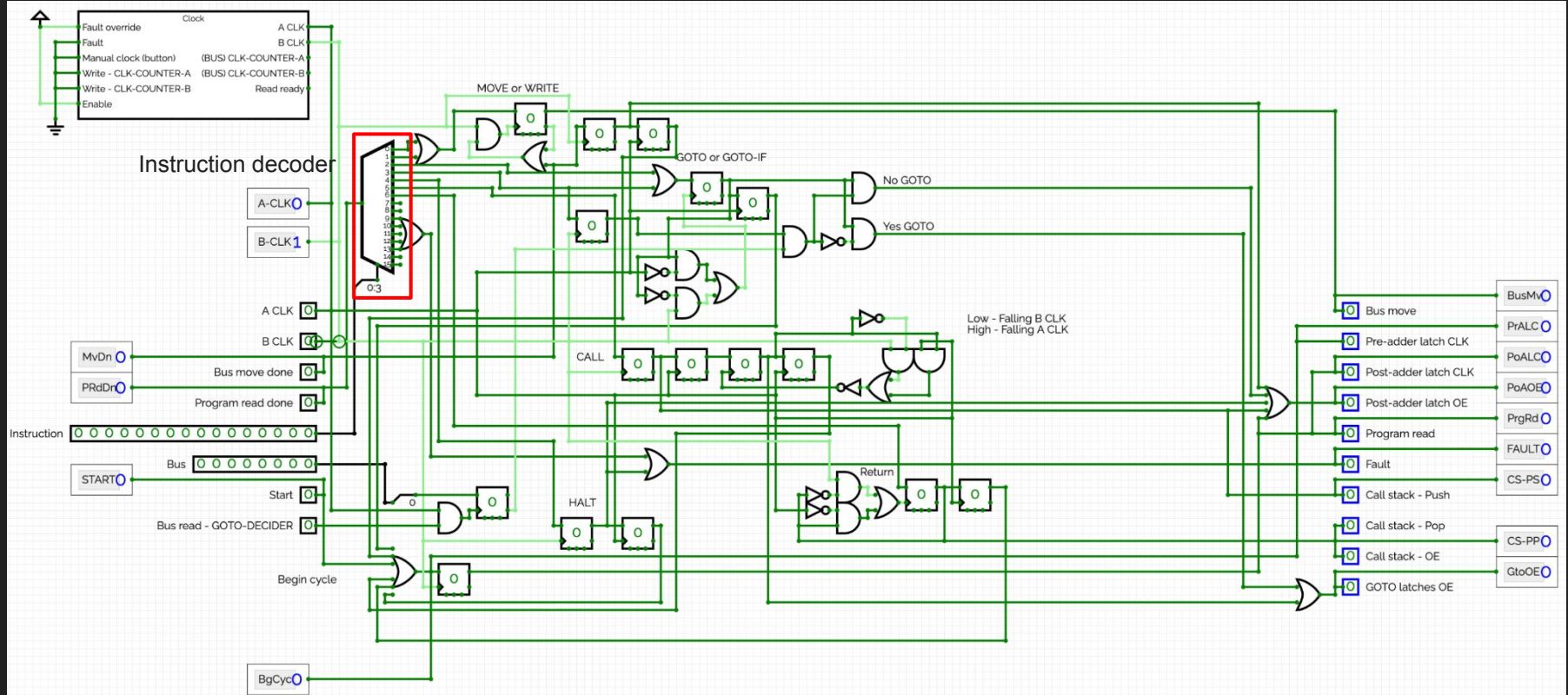
Timing

- Everything has to be synchronized by the clock
- Original clock scheme was stupid but too late to change it
- All the flashy lights do have a purpose



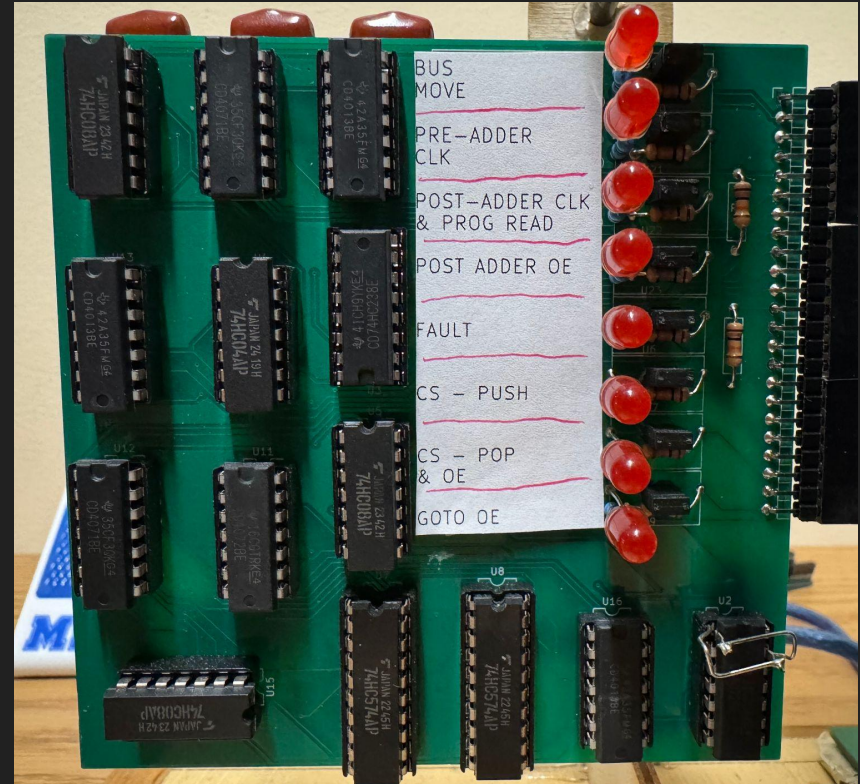
Timing diagrams VERY IMPORTANT

Main instruction decoder



Main instruction decoder (IRL)

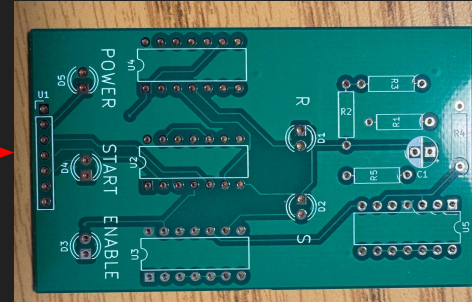
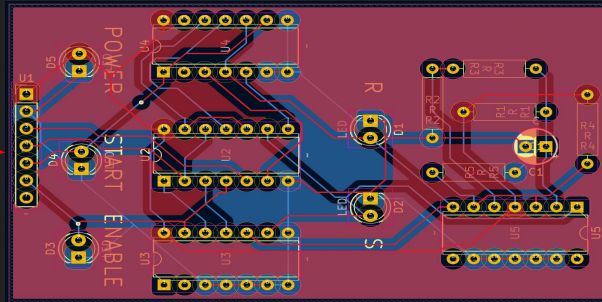
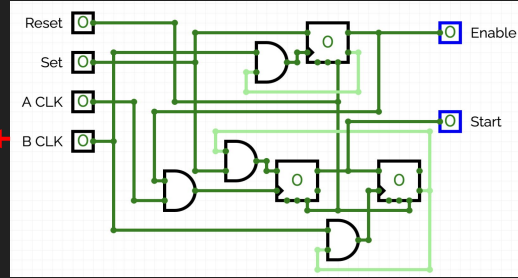
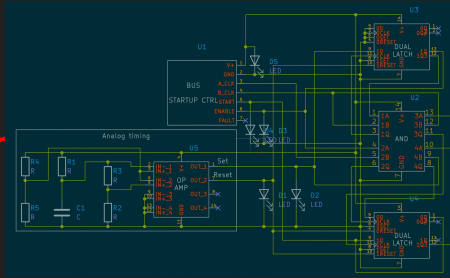
- For each instruction, tells the rest of the computer what to do
- Originally attached backwards, somehow did not get damaged???





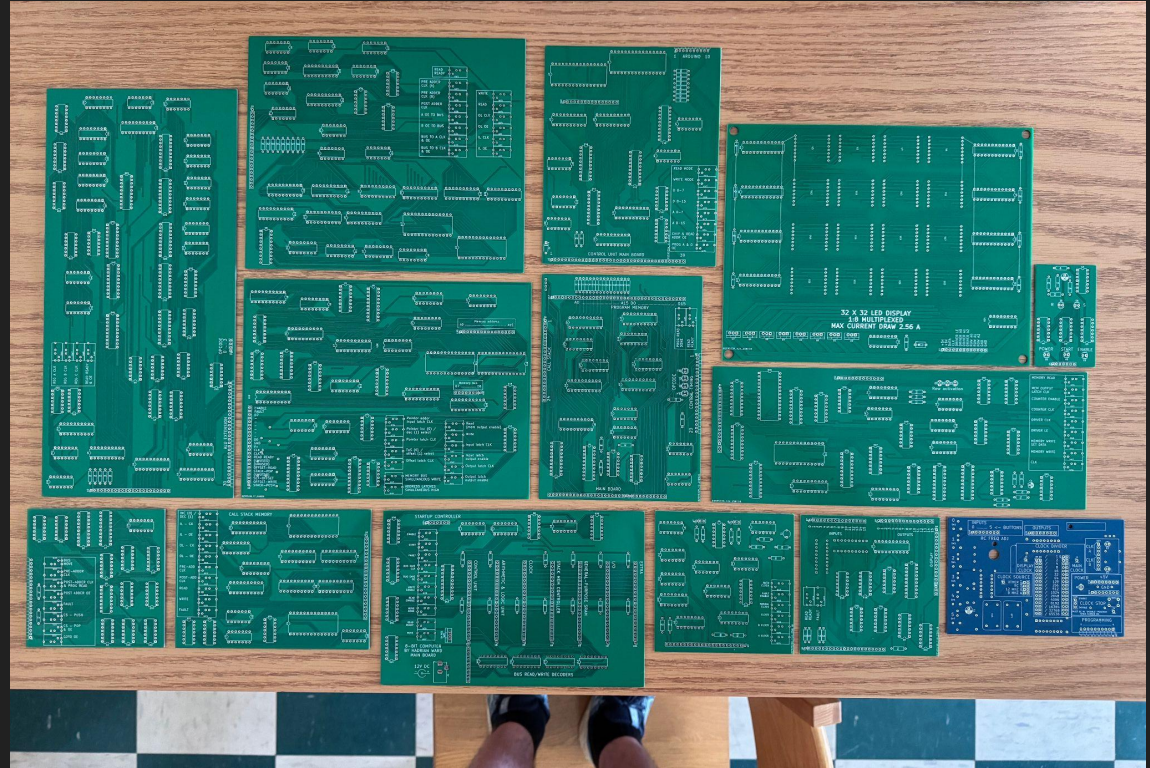
Designing Printed Circuit Boards (PCBs)

- Autorouting made it easy
- Transfer from Circuitverse exactly



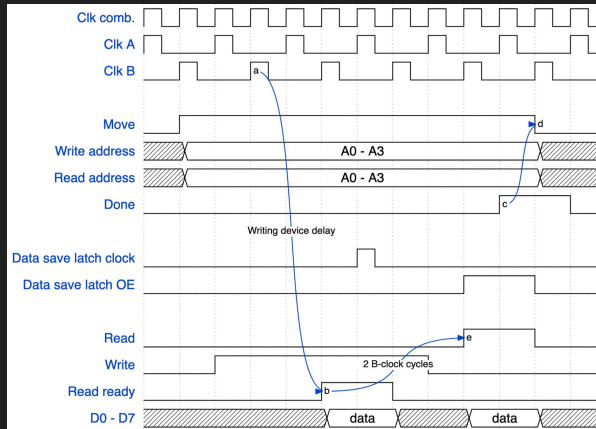
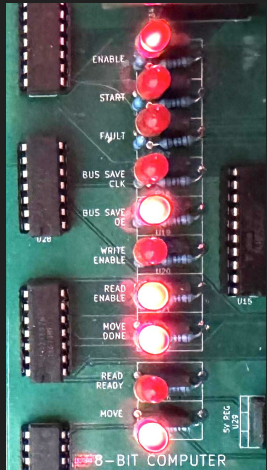
Soldering (and more soldering)

- I didn't think of taking many pictures at the time
- Used up most of a spool of solder
- > 4,200 individual solder joints
- Started around the beginning of September

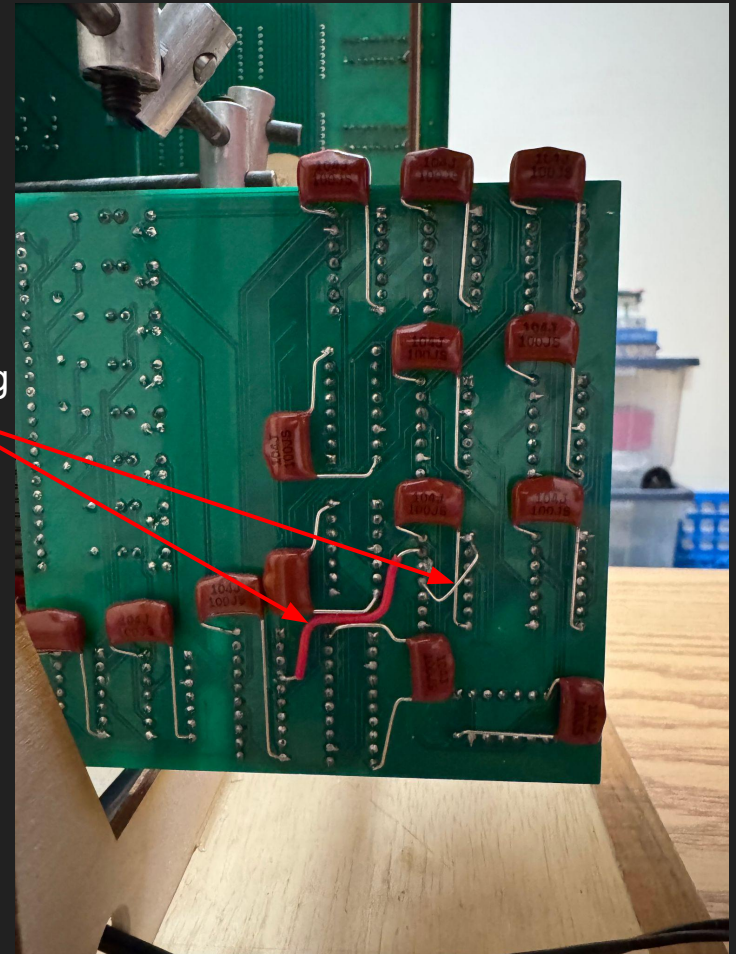


Troubleshooting

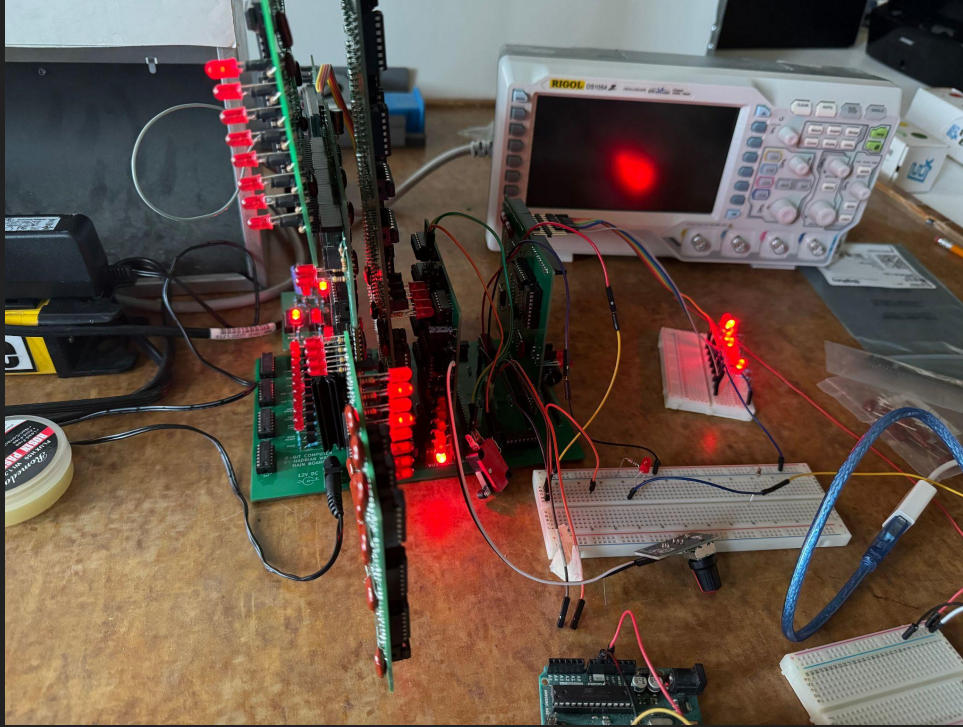
- 40% soldering, 60% debugging
- Timing signals have to work



Fixing goofy wiring



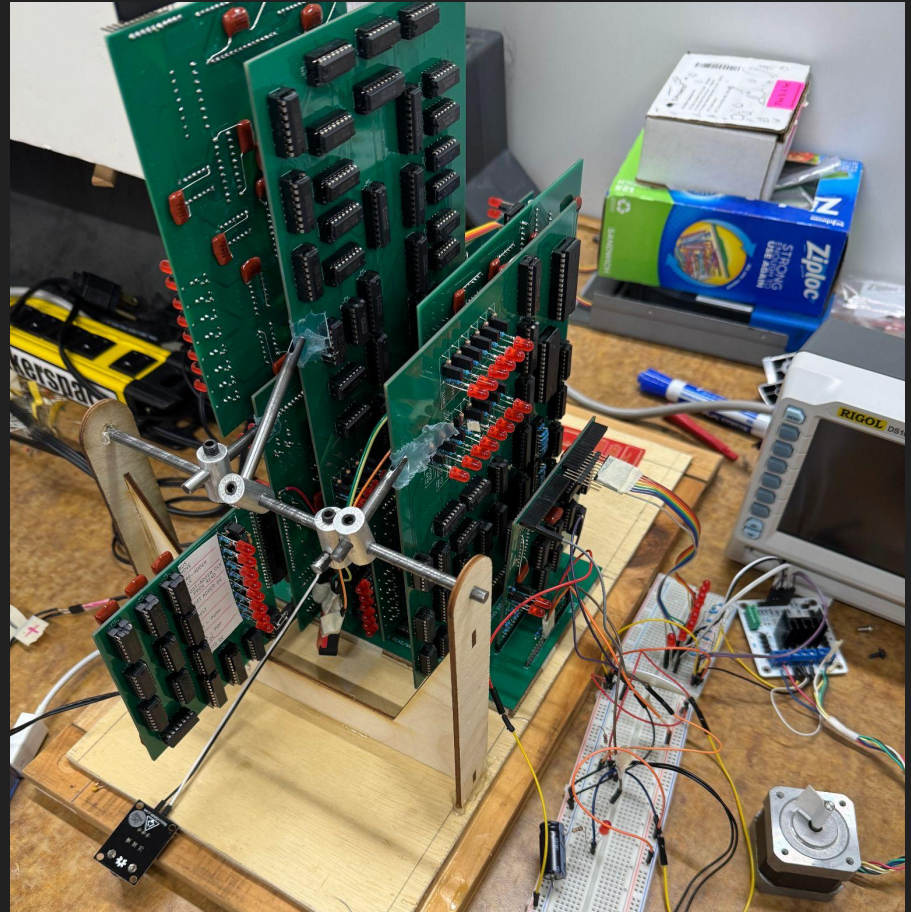
Coming together!



- Could run programs in a limited way
- Not structurally sound

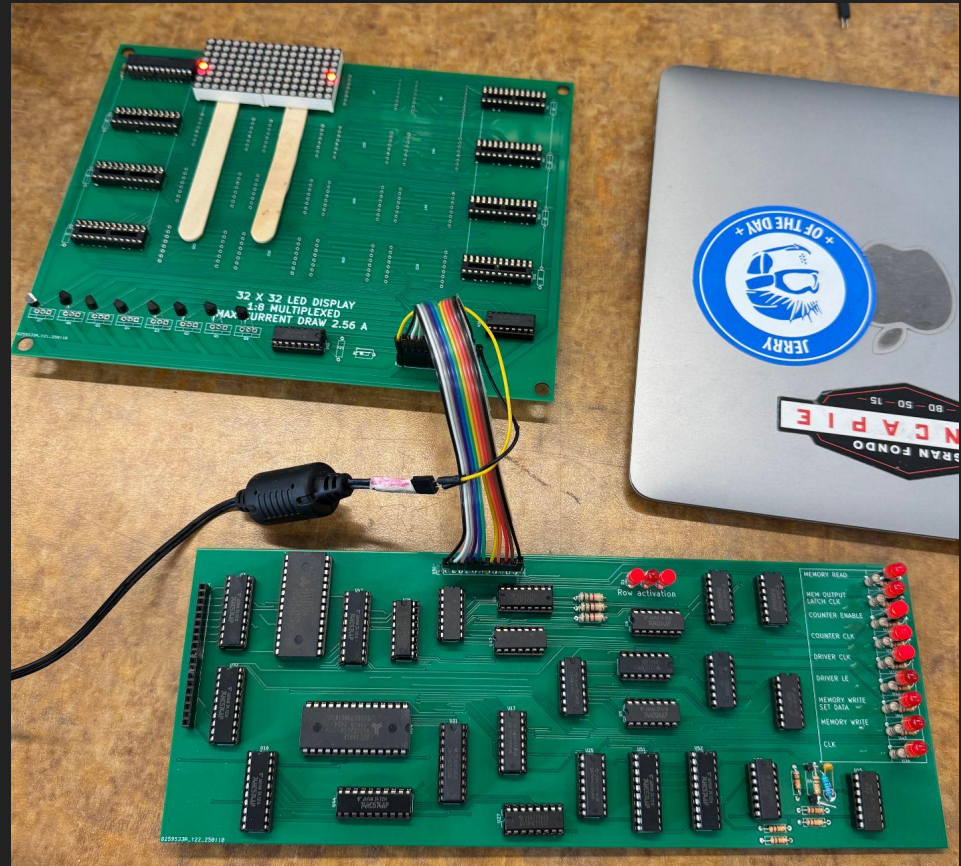
Original design complete

- December 19th
- Needed structural supports



Display

- Just some LEDs is boring
- A lot more LEDs
- Multiplexing
- Control circuit actually complicated
- Race condition, the most confusing problem yet



Programming

I did not have time to cover this as much as I wanted

- Emulator (simulate it on my laptop)
- Assembler, simple compiler
- Tetris took 2,000 lines of assembly



Specs (extremely impressive WOW!)

RAM: 64 kB, 128 kB counting the stack

Storage: 128 kB (only for program, 64 k instructions)

Clock speed: At least 187,500 Hz, maybe higher?

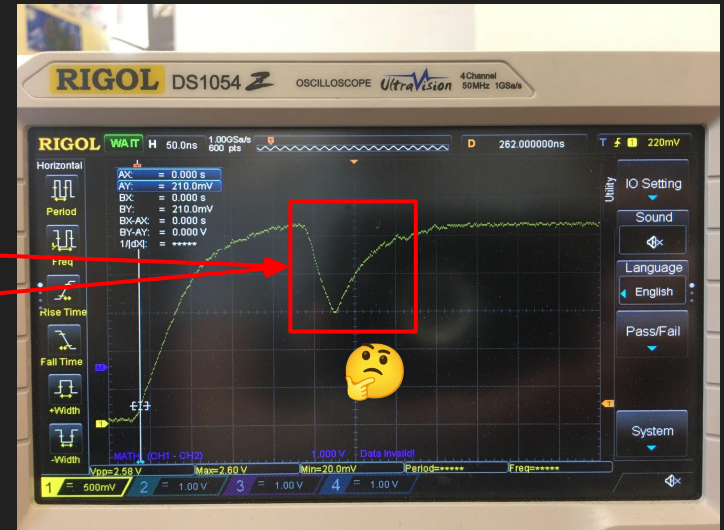
Screen size: 32 x 32 px

OS: None, it's like a microcontroller

Math: Integer addition / subtraction, bitwise operations

What I learned

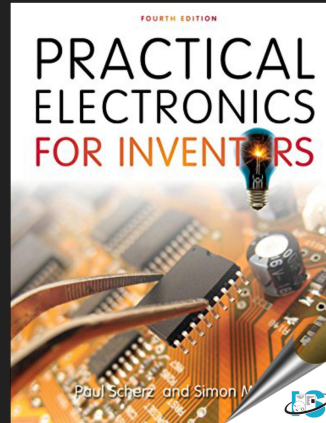
- Logic simulator $\neq$ physics in real world
- Capacitors across chip supplies
- I definitely want to do more of this
- Basic logic chips are very durable
- Think about mechanical constraints
- Programming in assembly weird
- Animating is fun but too much time



Acknowledgements

- Dr Hamlin
- Dr EG
- My Parents

Also this book I've had for ages:



Thank you for showing up

Questions?

No it cannot run Doom

Other questions?